

LegalEase – Complete Python Source Code

AI-Powered Legal Document Generator

main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routes import router
app = FastAPI(title="LegalEase API")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(router)
@app.get("/")
def root():
    return {"message": "LegalEase API is running"}
```

routes.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator
router = APIRouter()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_document(request: DocumentRequest):
    try:
        generator = GeminiDocumentGenerator()
        result = generator.generate(
            document_type=request.document_type,
            parties=request.parties,
            terms=request.terms,
            dates=request.dates
        )
        return {"document": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

ai_core/___init___py

ai_core/gemini_generator.py

```
import os
from dotenv import load_dotenv
import google.generativeai as genai
load_dotenv()
class GeminiDocumentGenerator:
    def __init__(self):
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError("GEMINI_API_KEY is not configured.")
        genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel("gemini-1.5-pro")
    def generate(self, document_type, parties, terms, dates):
        prompt = f"""
Create a formal legal document draft.
Document Type:
{document_type}
Parties:
{parties}
Terms:
{terms}
Dates:
{dates}
Requirements:
- Use clear formal legal language.
- Organize the document with suitable headings and clauses.
- Include the provided parties, dates and terms.
- Do not invent important personal or financial facts.
- Return only the document draft.
- Add a short disclaimer that this is an AI-generated draft
  and should be reviewed by a qualified legal professional.
"""
        response = self.model.generate_content(prompt)
        return response.text
```

[generators/__init__.py](#)

generators/document_utils.py

```
import re
from io import BytesIO
from docx import Document
from docx.shared import Inches, Pt
from docx.enum.text import WD_ALIGN_PARAGRAPH
from fpdf import FPDF

def sanitize_text(text: str) -> str:
    text = text or ""
    return re.sub(r"[^\x00-\x7F]+", " ", text)

def format_html_preview(text: str) -> str:
    safe = text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
    return safe.replace("\n", "<br>")

def format_docx(text: str, logo_path: str | None = None) -> bytes:
    doc = Document()
    if logo_path:
        try:
            paragraph = doc.add_paragraph()
            paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
            paragraph.add_run().add_picture(logo_path, width=Inches(1.2))
        except Exception:
            pass
    title = doc.add_paragraph()
    title.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = title.add_run("LegalEase Legal Document")
    run.bold = True
    run.font.name = "Times New Roman"
    run.font.size = Pt(16)
    for line in text.splitlines():
        p = doc.add_paragraph()
        p.paragraph_format.space_after = Pt(6)
        r = p.add_run(line)
        r.font.name = "Times New Roman"
        r.font.size = Pt(11)
    section = doc.sections[0]
    footer = section.footer.paragraphs[0]
    footer.alignment = WD_ALIGN_PARAGRAPH.CENTER
    footer.add_run("Generated by LegalEase")
    buffer = BytesIO()
    doc.save(buffer)
    return buffer.getvalue()

def format_pdf(text: str) -> bytes:
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=18)
    pdf.add_page()
    pdf.set_font("Times", "B", 16)
    pdf.cell(0, 10, "LegalEase Legal Document", ln=True, align="C")
    pdf.ln(5)
    pdf.set_font("Times", size=11)
    for line in sanitize_text(text).splitlines():
        if not line.strip():
            pdf.ln(4)
        else:
            pdf.multi_cell(0, 7, line)
    pdf.set_y(-15)
    pdf.set_font("Times", size=8)
    pdf.cell(0, 10, "Generated by LegalEase", align="C")
    return bytes(pdf.output(dest="S"))
```

app.py

```
import streamlit as st
import requests
from generators.document_utils import format_docx, format_pdf
API_URL = "http://127.0.0.1:8000/generate"
st.set_page_config(
    page_title="LegalEase",
    page_icon="■■■",
    layout="wide"
)
st.title("■■■ LegalEase")
st.subheader("AI-Powered Legal Document Generator")
document_type = st.selectbox(
    "Document Type",
    [
        "Employment Contract",
        "Lease Agreement",
        "Non-Disclosure Agreement",
        "Service Agreement",
        "Partnership Agreement",
        "Other"
    ]
)
parties = st.text_area(
    "Parties",
    placeholder="Enter the names and roles of the parties..."
)
terms = st.text_area(
    "Terms",
    placeholder="Enter important terms separated by semicolons..."
)
dates = st.text_input(
    "Dates",
    placeholder="Enter effective date and other relevant dates..."
)
if st.button("Generate Document"):
    payload = {
        "document_type": document_type,
        "parties": parties,
        "terms": terms,
        "dates": dates
    }
    try:
        response = requests.post(API_URL, json=payload, timeout=120)
        response.raise_for_status()
        st.session_state["document"] = response.json()["document"]
    except Exception as e:
        st.error(f"Generation failed: {e}")
if "document" in st.session_state:
    st.subheader("Editable Preview")
    edited = st.text_area(
        "Edit your document",
        value=st.session_state["document"],
        height=500
    )
    st.session_state["document"] = edited
    col1, col2, col3 = st.columns(3)
    with col1:
        st.download_button(
            "Download TXT",
            data=edited,
            file_name="LegalEase_Document.txt",
            mime="text/plain"
        )
    with col2:
        st.download_button(
            "Download DOCX",
            data=format_docx(edited),
            file_name="LegalEase_Document.docx",
            mime="application/vnd.openxmlformats-officedocument.wordprocessingml.document"
        )
    with col3:
        st.download_button(
            "Download PDF",
            data=format_pdf(edited),
            file_name="LegalEase_Document.pdf",
            mime="application/pdf"
        )
)
```

requirements.txt

```
fastapi
uvicorn
streamlit
python-docx
fpdf2
Pillow
requests
google-generativeai
python-dotenv
pydantic
```

.env.example

```
GEMINI_API_KEY=your_gemini_api_key_here
```

.gitignore

```
.env  
__pycache__/  
*.pyc  
.venv/  
venv/  
.idea/  
.vscode/
```

README.md

```
# LegalEase
LegalEase is an AI-powered legal document generator.
## Project Structure
- main.py - FastAPI application
- routes.py - API routes
- app.py - Streamlit frontend
- ai_core/gemini_generator.py - Gemini AI integration
- generators/document_utils.py - document export utilities
## Installation
1. Create a virtual environment.
2. Install dependencies:
    pip install -r requirements.txt
3. Create a .env file and add:
    GEMINI_API_KEY=your_gemini_api_key_here
## Run Backend
uvicorn main:app --reload
## Run Frontend
streamlit run app.py
Open the Streamlit URL shown in the terminal.
## Note
LegalEase generates AI-assisted document drafts. Documents should be reviewed by a qualified legal professional before use.
```